CS 349，2015 年夏季优化流水线处理器的性能 分配：6 月 6 日，截止：6 月 21 日，晚上 11:59

Harry Bovik (bovik@cs.cmu.edu) 是本作业的负责人。

1 简介

在本实验中，您将学习设计并实现一个流水线 Y86-64 处理器，并对其及基准程序进行优化以最大化性能。您可以对基准程序进行任何语义保持的转换，或者对流水线处理器进行改进，或者两者都进行。完成实验后，您将深刻理解代码与硬件之间的交互如何影响程序的性能。

实验室分为三个部分，每个部分都有自己的提交作业。在 A 部分，你需要编写一些简单的 Y86-64 程序，并熟悉 Y86-64 工具。在 B 部分，你将扩展 SEQ 模拟器以添加一条新指令。这两部分将为你准备 C 部分，实验室的核心，在那里你将优化 Y86-64 基准程序和处理器设计。

2 物流

你将独立完成这个实验。

任何关于作业的澄清和修订都将发布在课程网页上。

3 手册说明

站点特定：在此处插入一段说明，解释学生应该如何下载 archlab-handout.tar 文件。

1. 首先将文件 archlab-handout.tar 复制到您计划进行工作的（受保护）目录中。

2. 然后输入以下命令：tar xvf archlab-handout.tar。这将导致以下文件被解压到目录中：README、Makefile、sim.tar、archlab.pdf 和 simguide.pdf。

3. 然后，输入命令 tar xvf sim.tar。这将创建名为 sim 的目录，其中包含您的 Y86-64 工具副本。您将在该目录内完成所有工作。

4. 最后，切换到 sim 目录并构建 Y86-64 工具：unix> cd sim

```
unix> make clean; make
```

4 第一部分

您将在本部分中在目录 sim/misc 下工作。

您的任务是编写并模拟以下三个 Y86-64 程序。这些程序所需的行为由 examples.c 中的示例 C 函数定义。请确保在每个程序的开头添加您的姓名和学号。您可以通过首先使用 YAS 程序汇编它们，然后使用指令集模拟器 YIS 运行它们来测试您的程序。

在您的所有 Y86-64 函数中，您应遵循 x86-64 约定来传递函数参数、使用寄存器和使用栈。这包括保存和恢复您使用的任何被调用者保留寄存器。

sum.ys: 递归求链表元素之和

编写一个 Y86-64 程序 sum.ys，该程序迭代地求链表元素的和。你的程序应包含一些设置堆栈结构的代码，调用一个函数，然后停止。在这种情况下，该函数应该是与图 1 中的 C sum list 函数功能等效的 Y86-64 代码。使用以下三个元素的列表测试你的程序：

```
# 示例链表 .align 8 ele1:

        .quad 0x00a
        .quad ele2
ele2:
        .quad 0x0b0
        .quad ele3
ele3:
        .quad 0xc00
        .quad 0
```

```c
/* 链表元素 */
typedef struct ELE {
	长值;
	结构体 ELE *next;
} *list_ptr;

/* sum_list - 计算链表元素的总和 */
长 sum_list(list_ptr ls)
{
	长 val = 0;
	while (ls) {
		val += ls->val;
		ls = ls->next;
	}
	return val;
}

/* rsum_list - Recursive version of sum_list */
long rsum_list(list_ptr ls)
{
	如果 (!ls)
		返回 0;
	否则 {
		long val = ls->val;
		长休息 = rsum_list(ls->next);
		返回 val + 休息;
	}
}

/* 复制块 - 将源复制到目标并返回源异或校验和 */
long copy_block(long *src, long *dest, long len)
{
	长结果 = 0;
	while (长度 > 0) {
		long val = *src++;
		*dest++ = val;
		结果 ^= val;
		长度--;
	}
	返回结果;
}
```

图 1：Y86-64 解决方案函数的 C 版本。参见 sim/misc/examples.c

rsum.ys: 递归求链表元素之和

编写一个 Y86-64 程序 rsum.ys，该程序递归地求链表元素之和。此代码应类似于 sum.ys 中的代码，但应使用一个递归求和数字列表的 rsum list 函数，如图 1 中所示的 C 函数 rsum list。使用与测试 list.ys 相同的三个元素列表测试你的程序。

copy.ys: 将源块复制到目标块

编写一个程序（copy.ys），该程序将内存一部分的单词块复制到另一部分（非重叠区域）的内存中，计算所有复制的单词的校验和（异或）。
您的程序应包含设置堆栈帧、调用复制块函数然后停止的代码。该函数应与图 1 中所示的 C 语言复制块函数功能相同。请使用以下三个元素的源块和目标块测试您的程序：

```
.align 8
# 源块
src:
        .quad 0x00a
        .quad 0x0b0
        .quad 0xc00

# 目标块
dest:
        .quad 0x111
        .quad 0x222
        .quad 0x333
```

5 第二部分

在本部分中，您将在 sim/seq 目录下工作。

在第二部分中，您的任务是扩展 SEQ 处理器以支持 iaddq 指令，该指令在作业问题 4.51 和 4.52 中描述。为了添加这些指令，您需要修改 seq-full.hcl 文件，该文件实现了 CS:APP3e 教科书中描述的 SEQ 版本。此外，它还包含了一些您在解决方案中需要的常量声明。

您的 HCL 文件必须以包含以下信息的头部注释开始：

- 您的姓名和 ID。
- 描述 iaddq 指令所需的计算。以 CS:APP3e 文本中图 4.18 中 irmovq 和 OPq 的描述为指南。

构建和测试您的解决方案

修改完 seq-full.hcl 文件后，您将需要根据此 HCL 文件构建一个新的 SEQ 模拟器（ssim）实例，然后对其进行测试：

- 构建新的模拟器。您可以使用 make 来构建新的 SEQ 模拟器：

  ```
  unix> make VERSION=full
  ```

  这将构建一个使用您在 seq-full.hcl 中指定的控制逻辑的 ssim 版本。为了节省输入，您可以在 Makefile 中将 VERSION=full 赋值。

- 在简单的 Y86-64 程序上测试您的解决方案。对于您的初始测试，我们建议在 TTY 模式下运行简单的程序，如 asumi.yo（测试 iaddq），并将结果与 ISA 模拟进行比较：

  ```
  unix> ./ssim -t ../y86-code/asumi.yo
  ```

  如果 ISA 测试失败，那么你应该通过在 GUI 模式下单步调试模拟器来调试你的实现：

  ```
  unix> ./ssim -g ../y86-code/asumi.yo
  ```

- 使用基准程序重新测试你的解决方案。一旦你的模拟器能够正确执行小程序，那么你就可以在../y86-code 中的 Y86-64 基准程序上自动测试它了：

  ```
  unix> (cd ../y86-code; make testssim)
  ```

  这将运行 ssim 在基准程序上，并通过比较生成的处理器状态与高级 ISA 模拟的状态来检查正确性。请注意，这些程序中没有任何一个测试添加的指令。你只是在确保你的解决方案没有为原始指令注入错误。有关详细信息，请参阅文件 ../y86-code/README。

- 执行回归测试。一旦你可以正确执行基准程序，那么你应该运行位于 ../ptest 的完整回归测试集。要测试除了 iaddq 以外的所有内容，请留下：

  ```
  unix> (cd ../ptest; make SIM=../seq/ssim)
  ```

  测试您对 iaddq 的实现：

  ```
  unix> (cd ../ptest; make SIM=../seq/ssim TFLAGS=-i)
  ```

关于 SEQ 模拟器的更多信息，请参阅 CS:APP3e 指南 Y86-64 处理器模拟器（simguide.pdf）讲义。

```
/*
 * ncopy - 将 src 复制到 dst，返回正整数的数量
 * 包含在 src 数组中。
 */
word_t ncopy(word_t *src, word_t *dst, word_t len)
{
      word_t count = 0;
word_t val;

      while (长度 > 0) {
            val = *src++;
            *dst++ = val;
            if (val > 0)
                  count++
            len--
      }
      返回 count
}
```

图 2：ncopy 函数的 C 语言版本。见 sim/pipe/ncopy.c。

6 第 C 部分

在本部分中，您将在 sim/pipe 目录下工作。

图 2 中的 ncopy 函数将一个 len 元素整型数组 src 复制到非重叠的 dst，并返回 src 中包含的正整数的数量。图 3 显示了 ncopy 的 Y86-64 基线版本。pipe-full.hcl 文件包含 PIPE 的 HCL 代码副本，以及常量值 IIADDQ 的声明。

您在第 C 部分的任务是修改 ncopy.ys 和 pipe-full.hcl，目标是使 ncopy.ys 运行尽可能快。

你将提交两个文件：pipe-full.hcl 和 ncopy.ys。每个文件都应该以以下信息的头部注释开始：

- 你的姓名和学号。
- 对你的代码进行高级描述。在每种情况下，描述你如何以及为什么修改了你的代码。

编码规范

你可以随意进行任何修改，但需遵守以下约束：

- 你的 ncopy.ys 函数必须适用于任意大小的数组。你可能想通过简单地编写 64 条复制指令来为 64 元素数组硬编码解决方案，但这不是一个好主意，因为我们将根据你在任意数组上的性能来评分你的解决方案。

```
##################################################################
# ncopy.ys - 复制 src 块中的 len 个单词到 dst。
# 返回 src 中包含的正数单词数（>0）。
#
请在此处填写您的姓名和 ID。
#
请描述您如何以及为什么修改了基线代码。
#
##################################################################
请勿修改此部分。
函数序言。
# %rdi = 源, %rsi = 目标, %rdx = 长度
ncopy:

##################################################################
# 你可以修改这部分
        # 循环头
        xorq %rax,%rax                  # count = 0;
        andq %rdx,%rdx                  # len <= 0?
        jle 完成                        如果是，则转到完成:

循环: mrmovq (%rdi), %r10 # 从源读取值...
        rmmovq %r10, (%rsi)             将其存储到 dst
        andq %r10, %r10                 val <= 0?
        jle Npos                        # if so, goto Npos:
        irmovq $1, %r10
        addq %r10, %rax                 # count++
Npos: irmovq $1, %r10
        subq %r10, %rdx # len--
        irmovq $8, %r10
        addq %r10, %rdi                 # src++
        addq %r10, %rsi                 # dst++
        andq %rdx,%rdx                  # 长度大于 0?
        jg Loop                         如果如此，则进入循环:
##################################################################
不要修改以下代码部分
函数尾声。
完成:
        返回
##################################################################
# 请将以下标签保留在函数末尾
结束:
```

图 3：ncopy 函数的 Y86-64 基线版本。参见 sim/pipe/ncopy.ys。

- 您的 ncopy.ys 函数必须使用 YIS 正确运行。这里的“正确”是指它必须正确地复制 src 块，并在(%rax)中返回正确的正整数数量。
- 您的 ncopy 文件汇编版本不得超过 1000 字节。您可以使用提供的脚本 check-len.pl 检查任何程序的长度，该脚本将 ncopy 函数嵌入其中：

```
unix> ./check-len.pl < ncopy.yo
```

- 您的 pipe-full.hcl 实现必须通过 ../y86-code 和 ../ptest 中的回归测试（不使用 -i 标志测试 iaddq）。

除了这些，如果您认为实现 iaddq 指令会有帮助，您可以自由实现。您可以对 ncopy.ys 函数进行任何语义保持的转换，例如重新排序指令、用单个指令替换指令组、删除一些指令和添加其他指令。您可能需要阅读 CS:APP3e 第 5.8 节关于循环展开的内容。

构建和运行您的解决方案

为了测试您的解决方案，您需要构建一个调用您的 ncopy 函数的驱动程序。我们已为您提供了 gen-driver.pl 程序，该程序可以生成适用于任意大小输入数组的驱动程序。例如，输入以下命令：

```
unix> make drivers
```

将构建以下两个有用的驱动程序：

- sdriver.yo：一个小的驱动程序，用于测试包含 4 个元素的数组上的 ncopy 函数。如果您的解决方案正确，那么在复制 src 数组后，该程序将在寄存器%rax 中停止并带有值为 2。

- ldriver.yo: 这是一个大型驱动程序，用于测试对包含 63 个元素的较大数组进行 ncopy 函数的操作。如果您的解决方案正确，那么在复制 src 数组后，该程序将停止，寄存器%rax 中的值为 31（0x1f）。

每次您修改您的 ncopy.ys 程序，都可以通过输入以下命令重新构建驱动程序：

```
unix> make drivers
```

每次您修改您的 pipe-full.hcl 文件，都可以通过输入以下命令重新构建模拟器：

```
unix> make psim VERSION=full
```

如果你想重新构建模拟器和驱动程序，请输入

```
unix> make VERSION=full
```

要在 GUI 模式下测试你的解决方案，并在一个包含 4 个元素的数组上进行测试，请输入

```
unix> ./psim -g sdriver.yo
```

为了在更大的 63 元素数组上测试你的解决方案，请输入

```
unix> ./psim -g ldriver.yo
```

一旦你的模拟器正确运行了你在这两个块长度上的 ncopy.ys 版本，你将想要执行以下附加测试：

- 在 ISA 模拟器上测试您的驱动文件。请确保您的 ncopy.ys 函数能够正常工作。
  使用 YIS 进行测试：

  ```
  unix> make drivers unix> ../misc/yis
  sdriver.yo
  ```

- 使用 ISA 模拟器在一系列块长度上测试您的代码。Perl 脚本 correctness.pl 生成具有从 0 到某个限制（默认为 65）以及一些更大大小的块长度的驱动文件。它通过默认的 YIS 进行模拟，并检查结果。它生成一个报告，显示每个块长度的状态：

  ```
  unix> ./correctness.pl
  ```

  此脚本生成测试程序，其结果计数在每次运行中随机变化，因此它比标准驱动程序提供了更严格的测试。如果您在某些长度 K 上得到不正确的结果，您可以生成一个包含检查代码的驱动程序文件，其中结果随机变化：

  ```
  unix> ./gen-driver.pl -f ncopy.ys -n K -rc > driver.ys unix> make driver.yo
  unix> ../misc/yis driver.yo
  ```

  程序将以寄存器%rax 具有以下值结束：

  0xaaaa : 所有测试通过。

  0xbbbb : 计数不正确。

  0xcccc : 函数 ncopy 长度超过 1000 字节。

  0xdddd : 部分源数据未复制到目标位置。

  0xeeee : 一些单词在目标区域之前或之后被损坏。

- 在基准程序上测试您的管道模拟器。一旦您的模拟器能够正确执行 sdriver.ys 和 ldriver.ys，您应该将其与../y86-code 中的 Y86-64 基准程序进行测试：

  ```
  unix> (cd ../y86-code; make testpsim)
  ```

这将在基准程序上运行 psim，并将结果与 YIS 进行比较。

- 使用广泛的回归测试测试您的管道模拟器。一旦您能正确执行基准程序，那么您应该使用../ptest 中的回归测试来检查它。例如，如果您的解决方案实现了 iaddq 指令，那么

```
unix> (cd ../ptest; make SIM=../pipe/psim TFLAGS=-i)
```

- 使用管道模拟器在一系列块长度上测试您的代码。最后，您可以在管道模拟器上运行您之前在 ISA 模拟器上所做的相同代码测试

```
unix> ./correctnesspl -p
```

7 评估

该实验室值 190 分：A 部分 30 分，B 部分 60 分，C 部分 100 分。

A 部分

Part A 占 30 分，每个 Y86-64 解决方案程序 10 分。每个解决方案程序将根据正确性进行评估，包括对栈和寄存器的正确处理，以及与 examples.c 中的示例 C 函数的功能等效性。
如果评分者没有在 sum.ys 和 rsum.ys 程序中发现任何错误，则认为这两个程序是正确的，并且它们各自的 sum 列表和 rsum 列表函数将返回寄存器 %rax 中的和 0xcba。
如果评分者没有在 copy.ys 程序中发现任何错误，则认为该程序是正确的，并且 copy 块函数将返回寄存器 %rax 中的和 0xcba，将三个 64 位值 0x00a、0x0b 和 0xc 复制到以地址 dest 开始的 24 个字节中，并且不损坏其他内存位置。

Part B

这个实验室的部分值得 35 分：

- 描述 iaddq 指令所需的计算，得 10 分。
- 在 y86-code 中通过基准回归测试，得 10 分，以验证您的模拟器是否仍然正确执行基准测试套件。
- 通过 ptest 中的回归测试得 15 分，针对 iaddq。

第 C 部分

本实验的这一部分价值 100 分：如果你的 ncopy.ys 代码或修改后的模拟器在之前描述的任何测试中失败，你将不会获得任何分数。

ncopy.ys 和 pipe-full.hcl 的头部描述各 20 分，以及这些实现的品质。

- 60 分用于性能。要获得这里的分数，你的解决方案必须正确，如之前定义。也就是说，ncopy 在 YIS 上运行正确，pipe-full.hcl 在 y86-code 和 ptest 中通过所有测试。

  我们将以每元素周期（CPE）为单位来表示您函数的性能。也就是说，如果模拟代码需要 C 个周期来复制一个包含 N 个元素的块，那么 CPE 就是 C/N。PIPE 模拟器显示完成程序所需的总周期数。在标准 PIPE 模拟器上运行的 ncopy 函数的基本版本，使用一个包含 63 个元素的大的数组复制 63 个元素需要 897 个周期，因此 CPE 为 897/63=14.24。

  由于一些周期用于设置对 ncopy 的调用以及设置 ncopy 内部的循环，您会发现对于不同的块长度，您将得到不同的 CPE 值（通常随着 N 的增加，CPE 会下降）。因此，我们将通过计算从 1 到 64 个元素的块的平均 CPE 来评估您函数的性能。您可以使用 pipe 目录中的 Perl 脚本 benchmark.pl 来运行 ncopy.ys 代码的模拟，并计算平均 CPE。只需运行以下命令

```
unix> ./benchmark.pl
```

  看看会发生什么。例如，ncopy 函数的基线版本 CPE 值介于 29.00 和 14.27 之间，平均值为 15.18。请注意，此 Perl 脚本不会检查答案的正确性。请使用 script correctness.pl 进行此操作。

  您应该能够实现平均 CPE 小于 9.00。我们最好的版本平均为 7.48。如果您的平均 CPE 是 c，那么您在这个实验部分的得分 S 将为：

$$S = \begin{cases} 0, & c > 10.5 \\ 20 \cdot (10.5 - c), & 7.50 \leqslant c \leqslant 10.50 \\ 60, & c < 7.50 \end{cases}$$

  默认情况下，benchmark.pl 和 correctness.pl 编译并测试 ncopy.ys。使用 -f 参数指定不同的文件名。-h 标志会显示完整的命令行参数列表。

8 上交说明

站点特定：插入一个描述，说明学生应该如何提交实验的三个部分。以下是我们使用的 CMU 描述。

- 您将提交三组文件：
  - 第 A 部分：sum.ys、rsum.ys 和 copy.ys。
  - 第 B 部分：seq-full.hcl。

– 第 C 部分：ncopy.ys 和 pipe-full.hcl。

- 确保你在每个提交文件的顶部注释中包含了你的姓名和学号。
- 为提交部分 X 的文件，请进入您的 archlab-handout 目录，并输入：

```
unix> make handin-partX TEAM=teamname
```

  其中 X 为 a、b 或 c，teamname 为您的 ID。例如，要提交部分 A：

```
unix> make handin-parta TEAM=teamname
```

- 上交后，如果你发现错误并想提交修改后的副本，请输入

```
unix make handin-partX TEAM=teamname VERSION=2
```

  每次提交后，请继续增加版本号。

- 你可以通过查看来验证你的上交

```
CLASSDIR/archlab/handin-partX
```

  您在这个目录中拥有列表和插入权限，但没有读取或写入权限。

9 提示

- 按设计，sdriver.yo 和 ldriver.yo 都足够小，可以在 GUI 模式下调试。我们发现使用 GUI 模式调试最简单，并建议您使用它。
- 如果您在 Unix 服务器上以 GUI 模式运行，请确保已初始化 DISPLAY 环境变量：

```
unix> setenv DISPLAY myhost.edu:0
```

- 对于某些 X 服务器，当您在 GUI 模式下运行 psim 或 ssim 时，“程序代码”窗口最初可能以关闭的图标形式出现。只需单击图标即可展开窗口。
- 对于某些基于 Microsoft Windows 的 X 服务器，“内存内容”窗口不会自动调整大小。您需要手动调整窗口大小。
- 如果 psim 和 ssim 模拟器被要求执行一个不是有效的 Y86-64 目标文件的文件，它们将终止并出现段错误。